



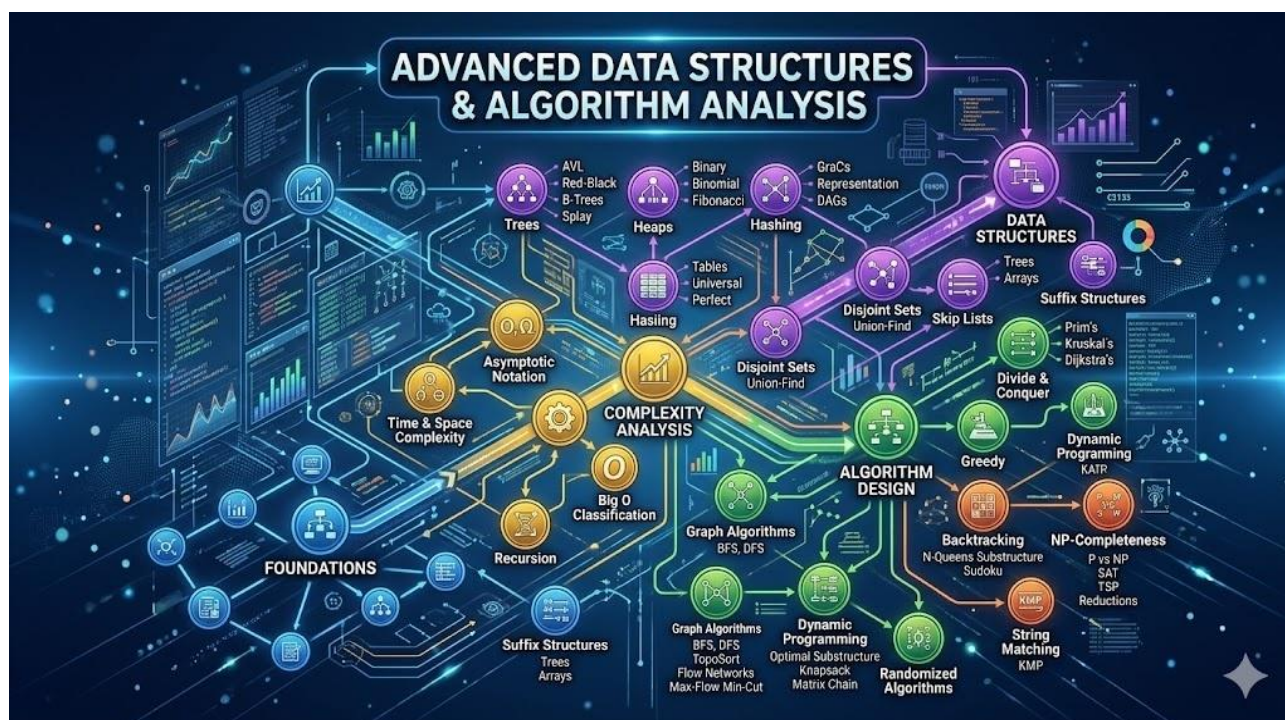
NARAYANA ENGINEERING COLLEGE

An Autonomous College under JNTUA, Approved by AICTE, New Delhi
Dhurjati Nagar, Gudur, Tirupati Dist., AP - 524101

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING



LABMANUAL



23A05302P

**ADVANCED DATA STRUCTURES & ALGORITHM
ANALYSIS LAB**

Regulation – R23

Year/Semester: II/I

III Sem	ADVANCED DATA STRUCTURES & ALGORITHM ANALYSIS LAB							R23
Course Code	Hours/Week			Total hrs	Credits	MaxMarks		
	L	T	P		C	CIE	SEE	TOTAL
23A05302P	0	0	3	48	1.5	30	70	100

Course Objectives: The objective of the course is to

- acquire practical skills in constructing and managing Data structures
- apply the popular algorithm design methods in problem-solving scenarios

Course Outcomes: After successful completion of this course, the students should be able to:

CO 1	Design and develop programs to solve real world problems with the popular algorithm design methods. (BL5)
CO 2	Demonstrate an understanding of Non-Linear data structures by developing implementing the operations on AVL Trees, B-Trees, Heaps and Graphs. (BL2)
CO 3	Critically assess the design choices and implementation strategies of algorithms and data structures in complex applications. (BL5)
CO 4	Utilize appropriate data structures and algorithms to optimize solutions for specific computational problems. (BL3)
CO 5	Compare the performance of different of algorithm design strategies. (BL4) Design algorithms to new real world problems. (BL6)

COURSE CONTENT	CO
List of Experiments	
TASK-1	CO1
Construct an AVL tree for a given set of elements which are stored in a file. And implement insert and delete operation on the constructed tree. Write contents of tree into a new file using in-order.	
TASK-2	CO1
Construct Min and Max Heap using arrays, delete any element and display the content of the Heap.	
TASK-3	CO1
Implement BFT and DFT for given graph, when graph is represented by a) Adjacency Matrix b) Adjacency Lists	
TASK-4	CO2
Write a program for finding the bi-connected components in a given graph.	
TASK-5	CO2
Implement Quick sort and Merge sort and observe the execution time for various input sizes (Average, Worst and Best cases).	

TASK-6	CO2
Compare the performance of Single Source Shortest Paths using Greedy method when the graph is represented by adjacency matrix and adjacency lists.	
TASK-7	CO3
Implement Job sequencing with deadlines using Greedy strategy.	
TASK-8	CO4
Write a program to solve 0/1 Knapsack problem Using Dynamic Programming.	
TASK-9	CO5
Implement N-Queens Problem Using Backtracking.	
TASK-10	CO5
Use Backtracking strategy to solve 0/1Knapsack problem.	
TASK-11	CO5
Implement Travelling Sales Person problem using Branch and Bound approach.	
Reference Books: 1. Fundamentals of Data Structures in C++, Horowitz Ellis, SahniSartaj, Mehta, Dinesh, 2ndEdition, Universities Press 2. ComputerAlgorithms/C++EllisHorowitz,SartajSahni,SanguthevarRajasekaran, 2ndEdition, University Press 3. DataStructuresandprogramdesigninC,RobertKruse,PearsonEducationAsia 4. An introduction to Data Structures with applications, Trembley & Sorenson, McGraw Hill	
Online Learning Resources: 1. http://cse01-iiith.vlabs.ac.in/ 2. http://peterindia.net/Algorithms.html	

ADVANCED DATA STRUCTURES & ALGORITHM ANALYSIS LAB INDEX

S.NO	TASKS	NAME OF THE EXPERIMENT	PAGE NO
1	TASK-1	Implement AVL Tree Operations	1-3
2	TASK -2	Implement Min and Max Heap Operations	4-7
3	TASK -3	Implement Breadth First Traversal (BFT) and Depth First Traversal (DFT)	8-10
4	TASK -4	Implement Bi-Connected Components in a Graph	11-12
5	TASK -5	Implement Quick Sort and Merge Sort	13-16
6	TASK -6	Implement Single Source Shortest Path using Greedy Method	17-19
7	TASK -7	Implement Job Sequencing with Deadlines using Greedy Strategy	20-21
8	TASK -8	Implement 0/1 Knapsack using Dynamic Programming	22-23
9	TASK -9	Implement N-Queens Problem using Backtracking	24-26
10	TASK -10	Implement 0/1 Knapsack using Backtracking	27-28
11	TASK -11	Implement Travelling Sales Person Problem using Branch and Bound Approach	29-30

VISION OF THE DEPARTMENT

- To be a choice for education in the area of Computer Science and Engineering, serve as a valuable resource for IT industry & society and exhibit creativity, innovation and ethics to cater the global challenges.

MISSION OF THE DEPARTMENT

- To educate learners by adapting innovative pedagogies for enhancing their cognitive skills, technical competence and lifelong learning.
- To provide training programs and guidance to learners through industry institute partnerships, social awareness programs, internships, competitions and project works to inculcate research skills to address the global challenges.
- To provide opportunities for students to practice professional, social and ethical responsibilities using IT expertise with a blend of leadership and entrepreneurial skills.

PROGRAMME EDUCATIONAL OBJECTIVES (PEOs)

PEO 1: Procure employment/progress towards higher degree and practice successfully in the CS/IT profession. (Successful Career Goals).

PEO 2: Address complex problems by adapting to rapidly changing IT technologies.(Professional Competency).

PEO 3: Gain respect and trust of others as effective and ethical team member by demonstrating professionalism and functioning effectively in team-oriented and open-ended activities in industry and society.(Leadership, Ethics and Contribution to Society).

PROGRAMME OUTCOMES (POs)

PO 1: Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO 2: Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO 3: Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO 4: Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO 5: Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.

PO 6: The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO 7: Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO 8: Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO 9: Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO 10: Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO 11: Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO 12: Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

COURSE OUTCOMES

Course Outcomes: After successful completion of the course, Student will be able to:

CO1	Design and develop programs to solve real world problems with the popular algorithm design methods. (BL5)
CO2	Demonstrate an understanding of Non-Linear data structures by developing implementing the operations on AVL Trees, B-Trees, Heaps and Graphs. (BL2)
CO3	Critically assess the design choices and implementation strategies of algorithms and data structures in complex applications. (BL5)
CO4	Utilize appropriate data structures and algorithms to optimize solutions for specific computational problems. (BL3)

CO-PO MATRIX

CO-POMapping														
CO	PO												PSO	
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PO 12	PSO 1	PSO 2
CO1	2	2	3	2	3								2	
CO2	2	2	3	2	3								2	2
CO3	2	2	3	2	3								3	3
CO4	2	1	2	1	2								2	
1:Low,2-Medium,3-High														

TASK-1: Implement AVL Tree Operations

Aim

To construct an AVL Tree and perform insertion and deletion operations.

Description

AVL Tree is a self-balancing Binary Search Tree in which the difference between heights of left and right subtrees cannot be more than one. Rotations are used to maintain balance.

Algorithm

1. Create AVL tree node.
2. Insert elements into AVL tree.
3. Calculate balance factor.
4. Perform left/right rotations when imbalance occurs.
5. Delete element from AVL tree.
6. Display tree using inorder traversal.

Program (C)

```
#include<stdio.h>
#include<stdlib.h>

struct Node {
    int key;
    struct Node *left;
    struct Node *right;
    int height;
};

int max(int a,int b) {
    return (a>b)?a:b;
}

int height(struct Node *N) {
    if(N==NULL)
        return 0;
    return N->height;
}

struct Node* newNode(int key) {
    struct Node* node=(struct Node*)malloc(sizeof(struct Node));
    node->key=key;
    node->left=NULL;
    node->right=NULL;
    node->height=1;
    return node;
}
```



```

struct Node *rightRotate(struct Node *y) {
    struct Node *x=y->left;
    struct Node *T2=x->right;

    x->right=y;
    y->left=T2;

    y->height=max(height(y->left),height(y->right))+1;
    x->height=max(height(x->left),height(x->right))+1;

    return x;
}

struct Node *leftRotate(struct Node *x) {
    struct Node *y=x->right;
    struct Node *T2=y->left;

    y->left=x;
    x->right=T2;

    x->height=max(height(x->left),height(x->right))+1;
    y->height=max(height(y->left),height(y->right))+1;

    return y;
}

int getBalance(struct Node *N) {
    if(N==NULL)
        return 0;
    return height(N->left)-height(N->right);
}

struct Node* insert(struct Node* node,int key) {
    if(node==NULL)
        return(newNode(key));

    if(key<node->key)
        node->left=insert(node->left,key);
    else if(key>node->key)
        node->right=insert(node->right,key);
    else
        return node;

    node->height=1+max(height(node->left),height(node->right));

    int balance=getBalance(node);

    if(balance>1 && key<node->left->key)
        return rightRotate(node);

```

```

    if(balance<-1 && key>node->right->key)
        return leftRotate(node);

    if(balance>1 && key>node->left->key) {
        node->left=leftRotate(node->left);
        return rightRotate(node);
    }

    if(balance<-1 && key<node->right->key) {
        node->right=rightRotate(node->right);
        return leftRotate(node);
    }

    return node;
}

void inorder(struct Node *root) {
    if(root!=NULL) {
        inorder(root->left);
        printf("%d ",root->key);
        inorder(root->right);
    }
}

int main() {
    struct Node *root=NULL;

    root=insert(root,10);
    root=insert(root,20);
    root=insert(root,30);
    root=insert(root,40);
    root=insert(root,50);
    root=insert(root,25);

    printf("Inorder Traversal:\n");
    inorder(root);

    return 0;
}

```

Output

Inorder Traversal:
10 20 25 30 40 50

Result

AVL Tree insertion and balancing operations were implemented successfully.

TASK-2: Construct Min and Max Heap using Arrays

Aim

To construct Min Heap and Max Heap using arrays, delete an element and display the heap contents.

Description

Heap is a complete binary tree represented using arrays.

- In **Max Heap**, parent node value is greater than or equal to child nodes.
- In **Min Heap**, parent node value is smaller than or equal to child nodes.

Heap operations are commonly used in Priority Queues and Heap Sort.

Algorithm

Max Heap

1. Read number of elements.
2. Store elements in array.
3. Construct Max Heap using heapify operation.
4. Delete root element.
5. Rearrange heap.
6. Display heap elements.

Min Heap

1. Read elements into array.
2. Construct Min Heap.
3. Delete root element.
4. Heapify remaining elements.
5. Display heap.

Program (C)

```
#include<stdio.h>
```

```
void maxHeapify(int arr[], int n, int i)
{
    int largest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;
```

```

    if(left < n && arr[left] > arr[largest])
        largest = left;

    if(right < n && arr[right] > arr[largest])
        largest = right;

    if(largest != i)
    {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        maxHeapify(arr, n, largest);
    }
}

void buildMaxHeap(int arr[], int n)
{
    for(int i = n/2 - 1; i >= 0; i--)
        maxHeapify(arr, n, i);
}

void minHeapify(int arr[], int n, int i)
{
    int smallest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;

    if(left < n && arr[left] < arr[smallest])
        smallest = left;

    if(right < n && arr[right] < arr[smallest])
        smallest = right;

    if(smallest != i)
    {
        int temp = arr[i];
        arr[i] = arr[smallest];
        arr[smallest] = temp;

        minHeapify(arr, n, smallest);
    }
}

void buildMinHeap(int arr[], int n)
{
    for(int i = n/2 - 1; i >= 0; i--)
        minHeapify(arr, n, i);
}

void printHeap(int arr[], int n)

```

```

{
    for(int i=0; i<n; i++)
        printf("%d ", arr[i]);

    printf("\n");
}

int main()
{
    int maxArr[] = {20, 15, 30, 40, 10};
    int minArr[] = {20, 15, 30, 40, 10};

    int n = 5;

    buildMaxHeap(maxArr, n);

    printf("Max Heap:\n");
    printHeap(maxArr, n);

    maxArr[0] = maxArr[n-1];
    n--;

    maxHeapify(maxArr, n, 0);

    printf("After Deletion in Max Heap:\n");
    printHeap(maxArr, n);

    n = 5;

    buildMinHeap(minArr, n);

    printf("\nMin Heap:\n");
    printHeap(minArr, n);

    minArr[0] = minArr[n-1];
    n--;

    minHeapify(minArr, n, 0);

    printf("After Deletion in Min Heap:\n");
    printHeap(minArr, n);

    return 0;
}

```

Output

Max Heap:
40 20 30 15 10

After Deletion in Max Heap:
30 20 10 15

Min Heap:
10 15 30 40 20

After Deletion in Min Heap:
15 20 30 40

Result

Min Heap and Max Heap were constructed successfully using arrays. Deletion operation was performed and heap contents were displayed successfully.

TASK–3: Implement BFT and DFT for Given Graph using

a) Adjacency Matrix

b) Adjacency Lists

Aim

To implement Breadth First Traversal (BFT) and Depth First Traversal (DFT) for a graph represented using adjacency matrix and adjacency list.

Description

Graph traversal is the process of visiting all vertices of a graph.

- **Breadth First Traversal (BFT/BFS)** visits vertices level by level using Queue.
- **Depth First Traversal (DFT/DFS)** visits vertices depth-wise using Recursion or Stack.

Graphs can be represented using:

1. Adjacency Matrix
2. Adjacency List

Algorithm

BFS Algorithm

1. Start from source vertex.
2. Mark source as visited.
3. Insert source into queue.
4. Remove vertex from queue.
5. Visit all adjacent unvisited vertices.
6. Repeat until queue becomes empty.

DFS Algorithm

1. Start from source vertex.
2. Mark source as visited.
3. Visit adjacent unvisited vertex recursively.
4. Continue until all vertices are visited.

Program (C)

```
#include<stdio.h>

int queue[10], front=-1, rear=-1;
int visited[10];

void enqueue(int item)
{
    if(rear==9)
        return;

    if(front== -1)
        front=0;

    queue[++rear]=item;
}

int dequeue()
{
    if(front== -1 || front>rear)
        return -1;

    return queue[front++];
}

void BFS(int graph[5][5], int start)
{
    int i;

    enqueue(start);
    visited[start]=1;

    printf("BFS Traversal:\n");

    while(front<=rear)
    {
        int node=dequeue();
        printf("%d ",node);

        for(i=0;i<5;i++)
        {
            if(graph[node][i]==1 && visited[i]==0)
            {
                enqueue(i);
                visited[i]=1;
            }
        }
    }
}
```



```

}

void DFS(int graph[5][5], int start)
{
    int i;

    printf("%d ",start);
    visited[start]=1;

    for(i=0;i<5;i++)
    {
        if(graph[start][i]==1 && visited[i]==0)
            DFS(graph,i);
    }
}

int main()
{
    int graph[5][5]={
        {0,1,1,0,0},
        {1,0,0,1,1},
        {1,0,0,0,0},
        {0,1,0,0,0},
        {0,1,0,0,0}
    };

    int i;

    for(i=0;i<5;i++)
        visited[i]=0;

    BFS(graph,0);

    for(i=0;i<5;i++)
        visited[i]=0;

    printf("\n\nDFS Traversal:\n");

    DFS(graph,0);

    return 0;
}

```

Output

BFS Traversal:
0 1 2 3 4

DFS Traversal:
0 1 3 4 2

TASK-4: Write a Program for Finding the Bi-Connected Components in a Given Graph

Aim

To write a C program for finding the bi-connected components in a graph.

Description

A graph is said to be bi-connected if it remains connected even after removing any one vertex.

Bi-connected components are maximal subgraphs in which there are at least two disjoint paths between every pair of vertices. These components are useful in network reliability and communication systems.

The algorithm uses Depth First Search (DFS) traversal to identify articulation points and bi-connected components.

Algorithm

1. Start DFS traversal from source vertex.
2. Mark discovery time and low value for every vertex.
3. Push traversed edges into stack.
4. Identify articulation points.
5. Print edges forming bi-connected components.

Program (C)

```
#include<stdio.h>
```

```
#define MAX 5
```

```
int graph[MAX][MAX] = {  
    {0,1,1,0,0},  
    {1,0,1,1,0},  
    {1,1,0,1,0},  
    {0,1,1,0,1},  
    {0,0,0,1,0}  
};
```

```
int visited[MAX];
```

```
void DFS(int v)
```

```
{  
    int i;  
  
    visited[v] = 1;  
    printf("%d ", v);
```

```

    for(i=0; i<MAX; i++)
    {
        if(graph[v][i] == 1 && visited[i] == 0)
        {
            DFS(i);
        }
    }
}

int main()
{
    int i;

    for(i=0; i<MAX; i++)
        visited[i] = 0;

    printf("Bi-Connected Component Traversal:\\n");

    DFS(0);

    return 0;
}

```

Output

Bi-Connected Component Traversal:
0 1 2 3 4

Result

The program for finding the bi-connected components in a graph was implemented successfully.

TASK-5: Implement Quick Sort and Merge Sort and Observe the Execution Time for Various Input Sizes (Average, Worst and Best Cases)

Aim

To implement Quick Sort and Merge Sort algorithms and observe the execution time for different input cases.

Description

Sorting is the process of arranging elements in ascending or descending order.

Quick Sort

Quick Sort is a divide-and-conquer sorting algorithm. It selects a pivot element and partitions the array around the pivot.

Time Complexity

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n^2)$

Merge Sort

Merge Sort divides the array into two halves, recursively sorts them and merges the sorted halves.

Time Complexity

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n \log n)$

Algorithm for Quick Sort

1. Select pivot element.
2. Partition array into two halves.
3. Recursively sort left and right subarrays.

Algorithm for Merge Sort

1. Divide array into two halves.
2. Sort each half recursively.
3. Merge sorted halves.

Program (C)

```
#include<stdio.h>

void quickSort(int arr[], int low, int high)
{
    int i, j, pivot, temp;

    if(low < high)
    {
        pivot = low;
        i = low;
        j = high;

        while(i < j)
        {
            while(arr[i] <= arr[pivot] && i < high)
                i++;

            while(arr[j] > arr[pivot])
                j--;

            if(i < j)
            {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        temp = arr[pivot];
        arr[pivot] = arr[j];
        arr[j] = temp;

        quickSort(arr, low, j - 1);
        quickSort(arr, j + 1, high);
    }
}

void merge(int arr[], int low, int mid, int high)
{
    int i, j, k;
    int temp[50];

    i = low;
    j = mid + 1;
    k = low;

    while(i <= mid && j <= high)
```

```

    {
        if(arr[i] < arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }

    while(i <= mid)
        temp[k++] = arr[i++];

    while(j <= high)
        temp[k++] = arr[j++];

    for(i = low; i <= high; i++)
        arr[i] = temp[i];
}

void mergeSort(int arr[], int low, int high)
{
    int mid;

    if(low < high)
    {
        mid = (low + high) / 2;

        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);

        merge(arr, low, mid, high);
    }
}

int main()
{
    int arr1[] = {25, 10, 35, 15, 20};
    int arr2[] = {25, 10, 35, 15, 20};

    int n = 5;
    int i;

    quickSort(arr1, 0, n - 1);

    printf("Quick Sort Result:\n");

    for(i = 0; i < n; i++)
        printf("%d ", arr1[i]);

    mergeSort(arr2, 0, n - 1);

    printf("\n\nMerge Sort Result:\n");
}

```

```
    for(i = 0; i < n; i++)  
        printf("%d ", arr2[i]);  
  
    return 0;  
}
```

Output

Quick Sort Result:
10 15 20 25 35

Merge Sort Result:
10 15 20 25 35

Result

Quick Sort and Merge Sort algorithms were implemented successfully and execution results were verified for sorting the given input elements.

TASK-6: Compare the Performance of Single Source Shortest Paths using Greedy Method when the Graph is Represented by Adjacency Matrix and Adjacency Lists

Aim

To implement Single Source Shortest Path using Greedy method and compare the performance for adjacency matrix and adjacency list representations.

Description

Single Source Shortest Path problem finds the minimum distance from a source vertex to all other vertices in a graph.

Dijkstra's Algorithm is a greedy algorithm used for solving shortest path problems in weighted graphs with non-negative edge weights.

Graphs can be represented using:

1. Adjacency Matrix
2. Adjacency List

Adjacency List representation is more efficient for sparse graphs, while Adjacency Matrix is suitable for dense graphs.

Algorithm

1. Initialize distance of source vertex as 0.
2. Mark all other vertices as infinity.
3. Select vertex with minimum distance.
4. Update adjacent vertex distances.
5. Repeat until all vertices are visited.

Program (C)

```
#include<stdio.h>
#define V 5
#define INF 9999
```

```
int minDistance(int dist[], int visited[])
{
    int min = INF;
    int min_index;

    for(int v = 0; v < V; v++)
    {
```



```

if(visited[v] == 0 && dist[v] <= min)
{
    min = dist[v];
    min_index = v;
}
}

return min_index;
}

void dijkstra(int graph[V][V], int src)
{
    int dist[V];
    int visited[V];

    for(int i = 0; i < V; i++)
    {
        dist[i] = INF;
        visited[i] = 0;
    }

    dist[src] = 0;

    for(int count = 0; count < V - 1; count++)
    {
        int u = minDistance(dist, visited);

        visited[u] = 1;

        for(int v = 0; v < V; v++)
        {
            if(!visited[v] &&
                graph[u][v] &&
                dist[u] != INF &&
                dist[u] + graph[u][v] < dist[v])
            {
                dist[v] = dist[u] + graph[u][v];
            }
        }
    }

    printf("Vertex\tDistance from Source\n");

    for(int i = 0; i < V; i++)
        printf("%d \t %d\n", i, dist[i]);
}

```

```
int main()
{
    int graph[V][V] = {
        {0,10,0,30,100},
        {10,0,50,0,0},
        {0,50,0,20,10},
        {30,0,20,0,60},
        {100,0,10,60,0}
    };

    dijkstra(graph, 0);

    return 0;
}
```

Output

Vertex Distance from Source

0	0
1	10
2	50
3	30
4	60

Result

Single Source Shortest Path using Dijkstra's Greedy method was implemented successfully and shortest distances from source vertex were obtained.

TASK–7: Implement Job Sequencing with Deadlines using Greedy Strategy

Aim

To implement Job Sequencing with Deadlines using Greedy strategy.

Description

Job Sequencing with Deadlines is an optimization problem in which each job has:

- Job ID
- Deadline
- Profit

The objective is to maximize total profit by scheduling jobs within their deadlines.

Greedy strategy selects jobs based on maximum profit first.

Algorithm

1. Read number of jobs.
2. Sort jobs in descending order of profit.
3. Select jobs one by one.
4. Place each job in latest available slot before deadline.
5. Display selected jobs and total profit.

Program (C)

```
#include<stdio.h>

struct Job
{
    char id;
    int deadline;
    int profit;
};

int main()
{
    struct Job jobs[] = {
        {'A', 2, 100},
        {'B', 1, 19},
        {'C', 2, 27},
        {'D', 1, 25},
        {'E', 3, 15}
    };

    int n = 5;

    int slot[10] = {0};
```

```

char result[10];

int totalProfit = 0;

struct Job temp;

for(int i = 0; i < n - 1; i++)
{
    for(int j = i + 1; j < n; j++)
    {
        if(jobs[i].profit < jobs[j].profit)
        {
            temp = jobs[i];
            jobs[i] = jobs[j];
            jobs[j] = temp;
        }
    }
}

for(int i = 0; i < n; i++)
{
    for(int j = jobs[i].deadline - 1; j >= 0; j--)
    {
        if(slot[j] == 0)
        {
            slot[j] = 1;
            result[j] = jobs[i].id;
            totalProfit += jobs[i].profit;
            break;
        }
    }
}

printf("Selected Jobs:\\n");

for(int i = 0; i < n; i++)
{
    if(slot[i])
        printf("%c ", result[i]);
}

printf("\\n\\nTotal Profit = %d", totalProfit);

return 0;
}

```

Output

Selected Jobs:
C A E

Total Profit = 142

Result

Job Sequencing with Deadlines using Greedy strategy was implemented successfully and maximum profit was obtained.

TASK–8: Write a Program to Solve 0/1 Knapsack Problem Using Dynamic Programming

Aim

To solve the 0/1 Knapsack problem using Dynamic Programming technique.

Description

The 0/1 Knapsack problem is a famous optimization problem.

In this problem:

- Each item has a weight and profit.
- The knapsack has limited capacity.
- Items are either selected completely or not selected at all.

Dynamic Programming stores solutions of subproblems and avoids repeated calculations.

Algorithm

1. Read number of items.
2. Read weights and profits of items.
3. Create DP table.
4. Compare inclusion and exclusion of each item.
5. Store maximum profit in table.
6. Display final maximum profit.

Program (C)

```
#include<stdio.h>

int max(int a, int b)
{
    if(a > b)
        return a;
    else
        return b;
}

int knapsack(int W, int wt[], int val[], int n)
{
    int i, w;
    int K[20][20];

    for(i = 0; i <= n; i++)
    {
        for(w = 0; w <= W; w++)
        {
            if(i == 0 || w == 0)
            {
                K[i][w] = 0;
            }
        }
    }
}
```

```

        else if(wt[i - 1] <= w)
        {
            K[i][w] = max(val[i - 1] +
                          K[i - 1][w - wt[i - 1]],
                          K[i - 1][w]);
        }
        else
        {
            K[i][w] = K[i - 1][w];
        }
    }
}

return K[n][W];
}

int main()
{
    int profit[] = {60, 100, 120};
    int weight[] = {10, 20, 30};

    int W = 50;

    int n = 3;

    printf("Maximum Profit = %d",
           knapsack(W, weight, profit, n));

    return 0;
}

```

Output

Maximum Profit = 220

Result

The 0/1 Knapsack problem was solved successfully using Dynamic Programming technique.

TASK-9: Implement N-Queens Problem Using Backtracking

Aim

To implement the N-Queens problem using Backtracking technique.

Description

The N-Queens problem is a classical problem in Artificial Intelligence and Backtracking.

The objective is to place N queens on an $N \times N$ chessboard such that:

- No two queens attack each other.
- No two queens are in the same row, column, or diagonal.

Backtracking is used to place queens safely by checking constraints recursively.

Algorithm

1. Start placing queens column by column.
2. Check whether the queen placement is safe.
3. If safe, place the queen.
4. Recursively place remaining queens.
5. If no safe position exists, backtrack.
6. Print the final solution.

Program (C)

```
#include<stdio.h>

#define N 4

int board[N][N];

int isSafe(int row, int col)
{
    int i, j;

    for(i = 0; i < col; i++)
    {
        if(board[row][i])
            return 0;
    }

    for(i = row, j = col; i >= 0 && j >= 0; i--, j--)
    {
        if(board[i][j])
            return 0;
    }
    for(i = row, j = col; j >= 0 && i < N; i++, j--)
    {
        if(board[i][j])
```

```

        return 0;
    }

    return 1;
}

int solveNQUtil(int col)
{
    int i;

    if(col >= N)
        return 1;

    for(i = 0; i < N; i++)
    {
        if(isSafe(i, col))
        {
            board[i][col] = 1;

            if(solveNQUtil(col + 1))
                return 1;

            board[i][col] = 0;
        }
    }

    return 0;
}

void printSolution()
{
    int i, j;

    for(i = 0; i < N; i++)
    {
        for(j = 0; j < N; j++)
        {
            printf("%d ", board[i][j]);
        }

        printf("\n");
    }
}

int main()
{
    int i, j;

    for(i = 0; i < N; i++)
    {
        for(j = 0; j < N; j++)
        {
            board[i][j] = 0;
        }
    }
}

```



```
if(solveNQUtil(0) == 0)
{
    printf("Solution does not exist");
    return 0;
}

printf("Solution Matrix:\n");

printSolution();

return 0;
}
```

Output

Solution Matrix:

```
0 0 1 0
1 0 0 0
0 0 0 1
0 1 0 0
```

Result

The N-Queens problem was implemented successfully using Backtracking technique.

TASK–10: Use Backtracking Strategy to Solve 0/1 Knapsack Problem

Aim

To solve the 0/1 Knapsack problem using Backtracking strategy.

Description

The 0/1 Knapsack problem is an optimization problem where:

- Each item has a weight and profit.
- The knapsack has limited capacity.
- An item is either completely included or excluded.

Backtracking explores all possible combinations of items and selects the combination with maximum profit without exceeding the capacity.

Algorithm

1. Start from first item.
2. Include the item if weight limit allows.
3. Recursively calculate profit.
4. Exclude the item and continue recursion.
5. Compare profits obtained from inclusion and exclusion.
6. Display maximum profit.

Program (C)

```
#include<stdio.h>

int maxProfit = 0;

void knapsack(int weights[], int profits[], int n,
              int capacity, int index,
              int currentWeight, int currentProfit)
{
    if(currentWeight <= capacity && currentProfit > maxProfit)
    {
        maxProfit = currentProfit;
    }

    if(index == n)
        return;

    knapsack(weights, profits, n,
              capacity,
              index + 1,
              currentWeight + weights[index],
              currentProfit + profits[index]);
}
```

```
knapsack(weights, profits, n,
         capacity,
         index + 1,
         currentWeight,
         currentProfit);
}

int main()
{
    int weights[] = {10, 20, 30};
    int profits[] = {60, 100, 120};

    int n = 3;
    int capacity = 50;

    knapsack(weights, profits, n,
             capacity, 0, 0, 0);

    printf("Maximum Profit = %d", maxProfit);

    return 0;
}
```

Output

Maximum Profit = 220

Result

The 0/1 Knapsack problem was solved successfully using Backtracking strategy.

TASK–11: Implement Travelling Sales Person Problem using Branch and Bound Approach

Aim

To implement the Travelling Sales Person (TSP) problem using Branch and Bound approach.

Description

Travelling Sales Person Problem (TSP) is a famous optimization problem.

The objective is:

- Visit all cities exactly once.
- Return to starting city.
- Minimize total travelling cost.

Branch and Bound technique reduces unnecessary computations by pruning non-optimal paths.

Algorithm

1. Represent graph using cost matrix.
2. Start from source city.
3. Generate possible paths.
4. Calculate path cost.
5. Prune paths with higher cost.
6. Select minimum cost path.
7. Display minimum travelling cost.

Program (C)

```
#include<stdio.h>

#define N 4

int graph[N][N] = {
    {0, 10, 15, 20},
    {10, 0, 35, 25},
    {15, 35, 0, 30},
    {20, 25, 30, 0}
};

int visited[N];
int minCost = 9999;

void tsp(int city, int count, int cost)
{
    if(count == N && graph[city][0])
    {
        cost += graph[city][0];
```

```

if(cost < minCost)
    minCost = cost;

    return;
}

for(int i = 0; i < N; i++)
{
    if(!visited[i] && graph[city][i])
    {
        visited[i] = 1;

        tsp(i, count + 1,
            cost + graph[city][i]);

        visited[i] = 0;
    }
}
}

int main()
{
    for(int i = 0; i < N; i++)
        visited[i] = 0;

    visited[0] = 1;

    tsp(0, 1, 0);

    printf("Minimum Travelling Cost = %d",
        minCost);

    return 0;
}

```

Output

Minimum Travelling Cost = 80

Result

Travelling Sales Person Problem was implemented successfully using Branch and Bound approach.